

Smart contract audit

st0x.oracle - DIA

Content

Project description	3
Executive summary	3
Reviews	3
Security findings	6
**** Critical	6
*** High	6
H01 - Zero pauseTimeBefore Is Accepted, Leaving the Pre-Action Corporate-Action Window Unguarded	6
** Medium	7
M01 - Confiscation From the Wrapper Vault Drops the Share Price Discontinuously With No Corporate-Action Pause	7
* Low	8
L01 - Permissionless Proxy Minting Makes Beacon Membership and the Deployment Event Unusable as Authenticity Signals	8
L02 - No Upper Bounds on maxAge, pauseTimeBefore and pauseTimeAfter	9
Informational	10
Approach and methodology	11



Project description

The st0x.oracle project provides upgradeable oracle infrastructure for pricing ST0x tokenized equities across lending protocols. It is deployed through beacon proxies, allowing a shared implementation to be upgraded while preserving stable oracle addresses.

The DIA stack exposes Chainlink-compatible prices for ERC-4626 wtStock vault shares by combining DIA's underlying-equity price with the vault's live assets-per-share ratio. It enforces price freshness and automatically pauses reads around corporate actions to prevent mismatched pre-action prices and post-action vault valuations. Oracle proxies are deployed deterministically using CREATE2 salts derived from their configurations.

Executive summary

Type	Library
Languages	Solidity
Methods	Architecture Review, Manual Review, Unit Testing, Functional Testing, Automated Review
Documentation	README.md
Repositories	https://github.com/ST0x-Technology/st0x.oracle

Reviews

Date	Repository	Commit
13/08/26	st0x.oracle	a6d4a2951a84114149a25fd8d723cc51411cae36
26/08/26	st0x.oracle	4a7ae099454cd1e7a0dc3b1985be99f557295d53 tag sol-v0.1.1
31/08/26	st0x.oracle	466ba2d927c63b76a2b82c6b284149e2b78088e6

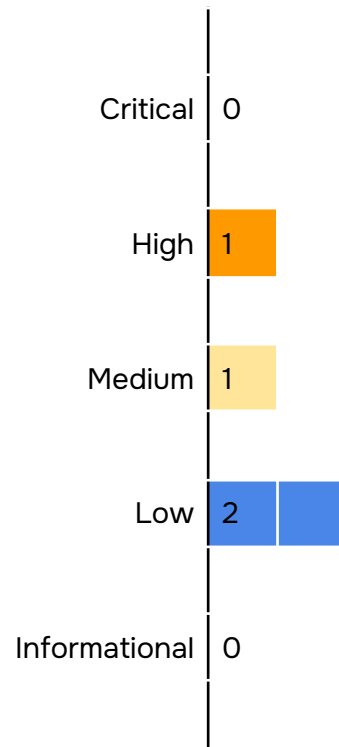


Scope

Contracts
src/interface/IAggregatorV2V3.sol src/interface/IDIAOracleV2.sol src/concrete/deploy/DIAVaultOracleBeaconSetDeployer.sol src/concrete/oracle/DIAVaultOracle.sol src/lib/LibDIAFeed.sol src/lib/LibCorporateActionsPause.sol



Technical analysis and findings



Security findings

**** Critical

No critical severity issue found.

*** High

H01 - Zero pauseTimeBefore Is Accepted, Leaving the Pre-Action Corporate-Action Window Unguarded

`DIAVaultOracle.initialize()` enforces a strict cross-epoch invariant on the post-action side – `pauseTimeAfter` must exceed `maxAge`, otherwise, `PauseTimeAfterBelowMaxAge` – but applies no equivalent constraint to `pauseTimeBefore`. The pause-coherence check only requires that the two windows are not both zero (`config.pauseTimeBefore == 0 && config.pauseTimeAfter == 0`). Because the subsequent check forces `pauseTimeAfter > maxAge >= 1`, `pauseTimeAfter` is always non-zero, so that disjunct can never fire on account of `pauseTimeBefore` alone.

Therefore, a configuration with `pauseTimeBefore = 0` and any valid `pauseTimeAfter` passes initialization, and the config is immutable thereafter. The share price is `diaPrice * totalAssets / totalSupply`, and both operands must belong to the same corporate-action epoch. A stock split's ex-date on the real market precedes the on-chain `effectiveTime` – the exchange sets the former, the latter by the operator – so the DIA feed can publish the already-halved equity price. At the same time, the vault's NAV ratio is still pre-split. Only the pre-window pauses reads across that interval; with `pauseTimeBefore = 0` nothing does.

Impact. On a 2:1 split, the oracle reports roughly half the true share price, so every `wtStock`-collateralised position appears undercollateralised and becomes liquidatable at a price the collateral does not actually trade at. This is the mirror of the post-action scenario the contract documents at length (a stale pre-action price against a rebalanced ratio, ~2x over-borrow), but the loss falls on borrowers rather than the protocol, and the asymmetry is not acknowledged anywhere. Because `DIAVaultOracle` has no admin and its config is immutable, there is no pause or parameter change available in the moment – remediation requires deploying a fresh proxy and migrating every consumer. Two aggravating factors: even a non-zero `pauseTimeBefore` is tied to nothing, since no invariant relates it to the ex-date lead time; and the upstream `scheduleCorporateAction` requires only `effectiveTime > block.timestamp`, so a short-notice schedule nullifies the pre-window regardless of how it is configured.



Path: src/concrete/oracle/DIAVaultOracle.sol

Recommendation: Replace the disjunction with two separate strict checks, rejecting `pauseTimeBefore == 0` as loudly as `pauseTimeAfter <= maxAge` is rejected today. Document the pre-action requirement symmetrically with the existing "Cross-epoch safety invariant" NatSpec block: `pauseTimeBefore` must exceed the interval between the market's ex-date revaluation and the on-chain `effectiveTime`. Since the oracle cannot enforce scheduling lead time, record a minimum notice period as an operational requirement on the corporate-action scheduler.

Found in: a6d4a29

Status: Fixed at 1025ebe

**** Medium**

M01 - Confiscation From the Wrapper Vault Drops the Share Price Discontinuously With No Corporate-Action Pause

The priced `wtStock` vault holds its backing as a plain token balance, and that balance is seizable by an upstream role. The chain is three steps:

1. `DIAVaultOracle._vaultSharePrice` computes $(\text{diaPrice} * \text{totalAssets} / \text{totalSupply})$, reading `totalAssets()` live on every call.
2. `StoxWrappedTokenVault` inherits OpenZeppelin's ERC-4626 default, so `totalAssets()` is `tStock.balanceOf(wtStockVault)` – a plain ERC-20 balance, not an internal accounting figure.
3. `OffchainAssetReceiptVault.confiscateShares(confiscatee, targetAmount, data)` transfers `tStock` out of any address named as `confiscatee`, the wrapper vault included, via a bare `_transfer()`.

So a confiscator seizing from the wrapper reduces `totalAssets()` in that transaction, and the share price falls by the same proportion in the very next oracle read.

The auto-pause cannot see this. `LibCorporateActionsPause` inspects only the corporate-action linked list, and a confiscation is a plain transfer – it creates no node, so there is no pre-window, no post-window and no `OraclePausedCorporateAction`. A stock split moves the same variable and is covered: it creates a node, and the whole cross-epoch machinery (`pauseTimeBefore` / `pauseTimeAfter` / the `pauseTimeAfter > maxAge` invariant) exists to bridge exactly that discontinuity. Confiscation moves `totalAssets` just as abruptly with none of that protection.

Impact. The resulting price is not wrong – after a seizure the vault genuinely does back fewer assets per share, so the lower figure is arithmetically correct. The problem is that



it arrives with zero warning: every `wtStock`-collateralised position can become liquidatable inside a single block, with no window in which a borrower could add collateral or repay, which is precisely the window the auto-pause was built to provide for the other discontinuous NAV move. The contract's own "Vault trust model" NatSpec analyses only the upward direction (donations, correctly concluded safe and value-additive) and never considers the downward one, even though `ICorporateActionsV1` explicitly warns integrators that confiscation exists as an RWA regulatory capability. This needs a privileged upstream role, so it is a trust-model and documentation gap rather than an exploitable bug.

Path: `src/concrete/oracle/DIAVaultOracle.sol`

Recommendation: The oracle cannot fix this alone – confiscation is upstream and must keep working, and a sanity band on the ratio would fail closed on legitimate moves, which the contract already argues against for good reason.

First, extend the "Vault trust model" NatSpec to cover the downward direction: state that any reduction of the wrapper's `tStock` balance outside the corporate-action list – confiscation being the concrete case – is a discontinuous price move that the auto-pause does not cover, and record it as an accepted risk that lending markets must size their liquidation parameters against.

Second, coordinate with `st0x.deploy` on an operational rule that a confiscation targeting a wrapper vault is preceded by a scheduled corporate action of any type inside the oracle's `actionTypeMask`.

Found in: `a6d4a295`

Status: Fixed at `617ce5d`

*** Low**

L01 - Permissionless Proxy Minting Makes Beacon Membership and the Deployment Event Unusable as Authenticity Signals

All three mint entry points – `newDIAVaultOracle`, `newST0xPriceOracle`, `newMorphoPairAdapter` – are plain external functions with no role check. The event NatSpec discusses only a front-run of the operator's intended config, correctly concluding that a config-identical mint grants no authority; a mint with a different, attacker-chosen config is not addressed.

Therefore, anyone can mint a proxy that is structurally indistinguishable from an official one – same governance-owned beacon, same implementation, `Deployment` emitted from the official deployer – but with arbitrary config. For `DIAVaultOracle` that means an attacker-controlled DIA feed and vault, which fully determine the price; for



ST0xPriceOracle the attacker holds both admin roles and the signer key on that instance.

Path: src/concrete/deploy/DIAVaultOracleBeaconSetDeployer.sol,
src/concrete/deploy/ST0xPriceOracleBeaconSetDeployer.sol,
src/concrete/deploy/MorphoPairAdapterBeaconSetDeployer.sol

Recommendation: Gate the mint functions behind an owner or allowlist, so that beacon membership becomes an authenticity signal and the documented CREATE2 front-running griefing vector closes with it. If permissionless minting is deliberate, state in the integrator section of the README that neither Deployment nor beacon membership authenticates an instance and that consumers must be wired from a published address list only. For the DIA stack, additionally validate `config.diaOracle` against the `DIA_FEED_BASE` constant already in `src/lib/LibDIAFeed.sol`.

Found in: a6d4a295

Status: Fixed at 5d3e018

L02 - No Upper Bounds on `maxAge`, `pauseTimeBefore` and `pauseTimeAfter`

The `initialize()` checks `maxAge` only for non-zero (`ZeroMaxAge`) and otherwise constrains it relatively, through `pauseTimeAfter > maxAge`; `pauseTimeBefore` and `pauseTimeAfter` have no upper bound beyond their `uint64` type. The `ST0xPriceOracle` contract does the opposite and states the reason: `ST0xPriceOracle` caps timeout at `MAX_TIMEOUT = 30 days`, reverting `TimeoutTooLarge`, because "a value larger than this is a misconfiguration (e.g. a milliseconds figure) that would silently disable staleness protection".

A relative invariant cannot catch that class of error, because it is scale-invariant: a units mistake scales both operands equally, and the check still passes. `maxAge = 7_200_000` with `pauseTimeAfter = 10_800_000` – "2 hours" and "3 hours" expressed in milliseconds, i.e. the exact values from the README's reference config – satisfies `pauseTimeAfter > maxAge` and initializes cleanly. The only implicit ceiling is that `pauseTimeAfter` is `uint64` and must exceed `maxAge`, capping the latter near `1.8e19` seconds, which is no practical constraint.

Path: `src/concrete/oracle/DIAVaultOracle.sol`

Recommendation: Add absolute limits – e.g., `MAX_AGE_LIMIT` and `MAX_PAUSE_WINDOW` constants with dedicated errors, mirroring `MAX_TIMEOUT` /



TimeoutTooLarge – and enforce them in initialize alongside the existing relative invariant.

Found in: a6d4a295

Status: Fixed at 0771727

Informational

No informational severity issue found.



Approach and methodology

To establish a uniform evaluation, we define the following terminology in accordance with the OWASP Risk Rating

Methodology:

	Likelihood Indicates the probability of a specific vulnerability being discovered and exploited in real-world scenarios
	Impact Measures the technical loss and business repercussions resulting from a successful attack
	Severity Reflects the comprehensive magnitude of the risk, combining both the probability of occurrence (likelihood) and the extent of potential consequences (impact)

Likelihood and impact are divided into three levels: High H, Medium M, and Low L. The severity of a risk is a blend of these two factors, leading to its classification into one of four tiers: Critical, High, Medium, or Low.

When we identify an issue, our approach may include deploying contracts on our private testnet for validation through testing. Where necessary, we might also create a Proof of Concept PoC to demonstrate potential exploitability. In particular, we perform the audit according to the following procedure:

	Advanced DeFi Scrutiny We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs
	Semantic Consistency Checks We then manually check the logic of implemented smart contracts and compare with the description in the white paper.
	Security Analysis The process begins with a comprehensive examination of the system to gain a deep understanding of its internal mechanisms, identifying any irregularities and potential weak spots.